# 30 Practical, Development-Focused Questions

## Regression (2)

1. **Q:** Using sklearn, build a **regularized linear regression** pipeline (with ColumnTransformer, StandardScaler, Ridge/Lasso) on a tabular dataset (e.g., sklearn.datasets.fetch_california_housing). What preprocessing, feature engineering, and hyperparameters will you implement, and how will you evaluate using **MSE, MAE, $R^2$** with **cross-validation**?
2. **Q:** Implement **polynomial regression** (with PolynomialFeatures + LinearRegression) and compare to a **tree-based regressor** (e.g., RandomForestRegressor) on the same dataset. How will you **detect overfitting** (learning curves, validation curves), and **choose the degree**/depth via GridSearchCV?

---

## Classification (2)

1. **Q:** Build a **binary classification** pipeline for sklearn.datasets.load_breast_cancer with **LogisticRegression** and **class weights**. How will you handle **feature scaling, stratified splitting**, and optimize the **decision threshold** to balance precision and recall? Show **ROC/PR curves**.
2. **Q:** Compare **SVM (with kernels)** vs **KNN** vs **Decision Tree** on sklearn.datasets.load_wine. Which model performs best across **macro/micro F1** and **confusion matrix** on a **multi-class** problem? Explain trade-offs and show **standardization impact**.

---

## Ensemble Learning (2)

1. **Q:** Train **RandomForest**, **GradientBoosting**, and **XGBoost** (if available) for tabular classification. How will you tune hyperparameters like n_estimators, max_depth, learning_rate and use **feature importance** or **permutation importance** to explain predictions?
2. **Q:** Implement a **stacking ensemble** combining logistic regression, SVM, and a tree model with a **meta-learner** (logistic regression). How will you avoid **data leakage** using StackingClassifier and nested cross-validation?

---

## Natural Language Processing (2)

1. **Q:** Build a **text classification** pipeline (e.g., sentiment) using **TF-IDF** + **LogisticRegression**. What will you do for **preprocessing** (tokenization, lowercasing, stopwords, lemmatization), and how will you handle **class imbalance** (resampling or class_weight)?
2. **Q:** Implement an **n-gram** feature pipeline (uni/bi/tri-grams) and evaluate the impact on performance using **AUC-PR** for imbalanced data. What regularization settings help when feature space explodes?

---

## NLTK (2)

1. **Q:** Using **NLTK**, build a preprocessing module: **sentence tokenization**, **word tokenization**, **stopword removal**, **POS tagging**, and **lemmatization (WordNet)**. How will you structure this module for reuse in multiple pipelines?
2. **Q:** Create a **Named Entity Recognition (NER)** baseline using nltk.ne_chunk on a news-like corpus. Compare entities extracted before and after **custom rule-based cleanup** (e.g., regex for email/URLs) and **domain stopwords**.

---

## Evaluation Metrics (2)

1. **Q:** Implement a **metrics dashboard** that, given predictions and labels, computes **Accuracy, Precision, Recall, F1 (macro/micro/weighted)**, **ROC-AUC**, **AUC-PR**, and plots **confusion matrix** and **calibration curves**. How will you design this for binary vs multi-class?
2. **Q:** For regression, implement **MSE, MAE, RMSE, $R^2$, MAPE**, and **residual analysis plots** (residual vs fitted, Q-Q plot). How will you detect **heteroscedasticity** and suggest fixes (e.g., **log-transform**, robust regression)?

---

## MLP Feed Forward Neural Network (2)

1. **Q:** Using **TensorFlow/Keras**, build an **MLP** for tabular classification with **BatchNormalization**, **Dropout**, and **EarlyStopping**. How will you choose the **number of layers/neurons**, optimizer (**Adam/RMSProp**), and implement **learning rate schedules**?
2. **Q:** Implement **MLP regression** on the California Housing dataset. Compare **MSE/$R^2$** to **LinearRegression** and **RandomForestRegressor**. What **normalization** and **feature scaling** choices affect performance in MLPs?

---

## Convolutional Neural Network (CNN) (2)

1. **Q:** Train a **CNN** on tf.keras.datasets.cifar10 for multi-class classification. Describe **data augmentation** (flip/rotate/zoom), **architecture choices** (Conv-BN-ReLU, MaxPool, GAP, Dense), and **regularization**. Evaluate with **Top-1/Top-5 accuracy**.
2. **Q:** Fine-tune a **pretrained CNN** (e.g., ResNet50) with **transfer learning**: freeze base layers, train classifier head, then unfreeze part of the base. How will you monitor **overfitting** and use **EarlyStopping + ReduceLROnPlateau**?

---

## Recurrent Neural Networks (RNNs) (2)

1. **Q:** Build a **Vanilla RNN** for **sequence classification** (e.g., IMDB sentiment) using embeddings + SimpleRNN. How will you handle **variable-length sequences** (padding/truncation), and compare **RNN vs MLP** baselines?
2. **Q:** Implement **gradient clipping** and **truncated BPTT** in an RNN for time series forecasting. Show how sequence length affects **training stability** and **performance**.

---

## Long-Short-Term-Memory Networks (LSTM) (2)

1. **Q:** Train an **LSTM** for **text sentiment** or **time series forecasting**. Compare **LSTM** to **GRU** and **1D CNN** baselines. How will you tune **dropout/recurrent_dropout**, **hidden units**, and **sequence length**?
2. **Q:** Build a **bidirectional LSTM** for sequence labeling (e.g., POS tagging). What improvements do you observe over unidirectional models, and how will you measure them (e.g., **token-level F1**)?

---

## Gated Recurrent Unit Networks (GRU) (2)

1. **Q:** Implement a **GRU-based** classifier with **embedding** and **global average pooling**, then compare training speed and performance vs LSTM on the same dataset. What's your conclusion on **parameter efficiency**?
2. **Q:** Add **attention** to a GRU model for text classification. How will you implement attention (e.g., Bahdanau-style) and measure its impact on **AUC-PR** for imbalanced data?

---

## Word Embeddings (2)

1. **Q:** Train **Word2Vec** (CBOW/Skip-gram) or use **FastText** embeddings with gensim for a custom corpus. How will you evaluate embeddings via **analogy tasks**, **nearest neighbors**, and downstream **classification accuracy**?
2. **Q:** Integrate **contextual embeddings** (e.g., **BERT**) via transformers for text classification. Compare **fine-tuning** vs **feature extraction** approaches. What are the trade-offs in **compute** and **accuracy**?

---

## Selection of Microsoft Azure AI Service (2)

1. **Q:** Given a **customer support chatbot** needing secure grounding on internal PDFs and websites, which Azure services will you choose? Explain selection of **Azure AI Search (Cognitive Search)** for indexing, **Azure OpenAI** for generation, and **Azure Content Safety** for moderation.
2. **Q:** For a **document processing pipeline** (invoices, receipts, ID cards) with validation rules and export to a database, which Azure service(s) fit best? Justify **Azure AI Document Intelligence**, **Azure Functions** for orchestration, and **Azure Storage/SQL** integration.

## Deploy Microsoft Azure AI Services (2)

1. **Q:** Design a **secure deployment** of an Azure OpenAI-backed API: use **Managed Identity** for auth, **Private Endpoints** in a **VNet**, **Key Vault** for secrets, and **Application Insights** for monitoring. How will you set **quotas** and handle **rate limiting**?
2. **Q:** Create a **CI/CD pipeline** (GitHub Actions or Azure DevOps) to build, test, and deploy an **Azure ML Online Endpoint**. What steps will you include (environment setup, unit tests, model registration, endpoint deployment, **blue/green** rollout, alerts)?

## Development & Implementation of Different Microsoft Azure AI Services (2)

1. **Q:** Implement a Python app that uses the **Azure AI Language** SDK (azure-ai-textanalytics) to perform **sentiment analysis** and **PII detection** on user messages, logs results to **Azure Monitor**, and stores summaries in **Cosmos DB**. What's your code structure?
2. **Q:** Build a **semantic search + chat** solution using **Azure AI Search + Azure OpenAI**: index PDFs from **Azure Blob Storage** with **indexers & skillsets**, perform **retrieval-augmented generation** (RAG) with grounding citations, and expose via **FastAPI**. How will you evaluate **response quality** and **latency**?